

NomadScholar AI

A Bilingual Multimodal RAG Assistant for Scholarship and University Applications Abroad

Student	Joe Sarkis
Course	Large Language Models - SP25-26
Date	2 June 2026
Repository	https://github.com/joeElieSarkis/nomadscholar-rag-assistant
Stack	React, FastAPI, LangChain, ChromaDB, Gemini, EasyOCR

Requirement Mapping

Requirement	Implementation
Knowledge base	10 curated official-source documents.
RAG	LangChain retrieval over ChromaDB with multilingual embeddings.
Conversation	Frontend sends previous turns to maintain multi-turn context.
Prompting	Role prompt, grounding rules, safety rules, and few-shot English/Arabic examples.
Multimodal	Image OCR; digital PDF text extraction added as an extension.
Structured output	Pydantic checklist schema for deadline, documents, eligibility, missing info, and next steps.
UI	React chat with uploads, sources, dark mode, edit/copy, selected-text replies, and checklist export.
Testing	26 documented test cases and 5 retrieval-improvement cases.

Project Overview & Goals

NomadScholar AI is a domain-specific virtual assistant for scholarship and university applications abroad. It is designed for Lebanese, Arab, and international students who need to understand official guidance, compare requirements, prepare documents, and avoid missing deadlines. The assistant answers conversationally in English or Arabic, explains uploaded application material, and turns admissions text into practical next steps.

The project is intentionally bounded for safety and reliability. It does not guarantee admission, scholarships, visas, or funding. Instead, it helps users understand retrieved information and reminds them that final decisions belong to universities, scholarship providers, embassies, and official portals.

Design Decisions

The knowledge base contains 10 curated documents from reusable official-source guidance, including Campus France, Common App, DAAD, EducationUSA, and Erasmus Mundus material. I kept the data set short and focused instead of scraping broadly, because reliable RAG depends more on source quality than on source quantity.

The backend uses FastAPI, LangChain, ChromaDB, HuggingFace multilingual embeddings, and Gemini. ChromaDB stores embedded chunks from the local data folder. The API returns source titles separately, allowing the frontend to display transparent source pills without forcing inline citations into the answer. I chose React and FastAPI rather than Gradio or Streamlit because the custom interface better demonstrates product-level UI development.

Prompting, Memory & Structured Output

The system prompt defines NomadScholar AI as a friendly admissions and scholarship guide. It requires same-language responses, factual grounding in retrieved context or uploaded text, no invented deadlines or eligibility rules, and clear reminders to verify official sources. Few-shot examples control tone for English guidance, Arabic guidance, refusal of guarantee claims, and explanation of recommendation-letter workflows.

Conversation memory is handled by sending previous user/assistant turns from the frontend to the backend. I also added selected-text replies: when a user highlights part of an answer and asks a follow-up, the selected excerpt is sent as explicit context. This makes follow-up questions more precise and closer to modern chat interfaces.

Structured output is implemented with a Pydantic ApplicationChecklist schema. The checklist extractor returns deadline, required documents, eligibility notes, missing information, and next steps. The frontend displays this as readable cards, raw JSON, and an optional PDF export.

Multimodal Interface

The required multimodal feature is image input. Users can upload PNG, JPG, or JPEG screenshots of requirements; OpenCV preprocessing improves readability and EasyOCR extracts text. The extracted text is then passed into the same RAG answer chain. As an extra feature, the system also supports digital PDFs with selectable text using PyPDF, because many admissions and scholarship instructions are distributed as PDF files.

The React UI includes multi-chat local history, source display, file attachment, message copy, edit/regenerate, stop generation, selected-text reply highlighting, dark mode, a checklist panel, and checklist PDF export. These additions are not all required, but they make the assistant feel more like a real application than only a backend demo.

Testing, Challenges & Future Work

Testing is documented in the tests folder. The project includes 26 cases covering in-scope English questions, Arabic questions, greetings, out-of-scope prompts, guarantee refusals, chat memory, selected-text replies, image upload, PDF upload, checklist extraction, and UI behavior. Five separate cases show how retrieval improves answers for DAAD, Campus France, Erasmus Mundus, EducationUSA, and Arabic scholarship-document questions.

The main challenge was preventing misleading answers. I added routing for greetings, small talk, guarantee questions, and out-of-scope topics so unrelated prompts do not show irrelevant sources. Another challenge was source quality, so the final knowledge base remained curated and official-source focused. Future improvements include speech-to-text and text-to-speech, scanned-PDF OCR, a larger knowledge base, live web retrieval for fresh deadlines, user accounts, deployment, and chunk-level citations beside specific claims.

Screenshot Evidence

Figures 1-4 demonstrate different capabilities required by the assignment and added product features: image-based OCR question answering, selected-text reply context for precise follow-ups, bilingual Arabic support, and structured checklist extraction. Digital PDF upload is also supported and documented in testing, but the screenshot evidence below focuses on the strongest visible UI features without mislabeling the reply screenshot as a PDF example.

Screenshot Evidence: Multimodal Input & Reply Feature

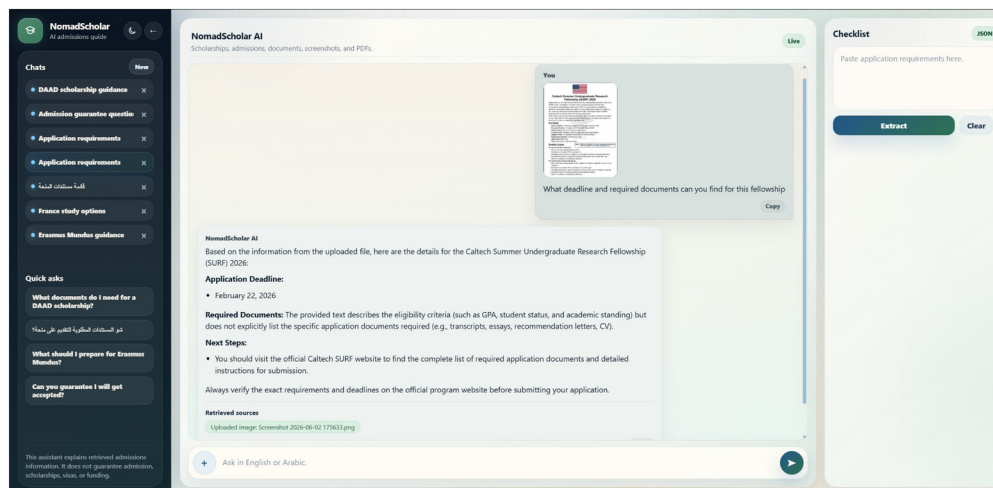


Figure 1. Image upload with OCR-based scholarship deadline and document guidance.

Image input: the assistant extracts text from an uploaded screenshot and returns a grounded answer with the uploaded file listed as a source.

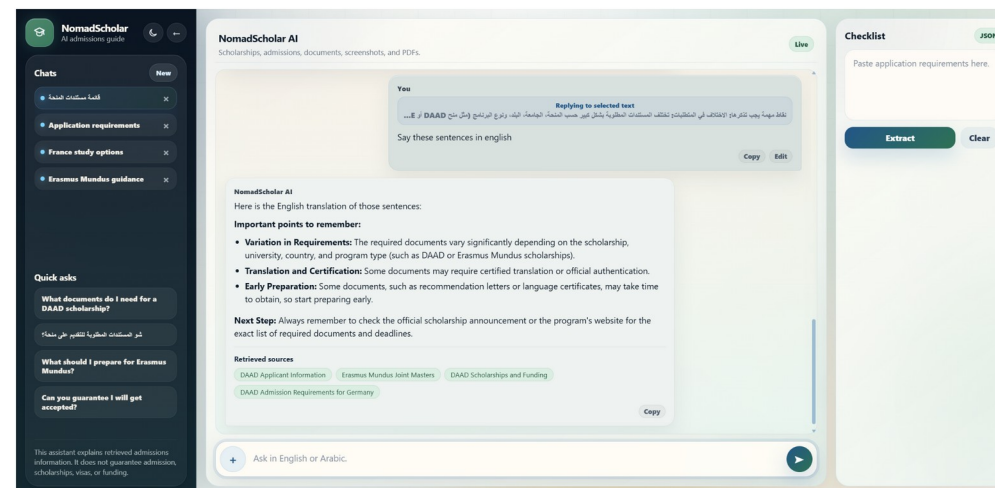


Figure 2. Context follow-up showing selected-text reply and grounded answer.

Contextual reply: selected text from a previous answer is attached to a follow-up question, allowing the assistant to answer that exact point rather than treating it as a separate generic request.

Screenshot Evidence: Bilingual Support & Structured Output

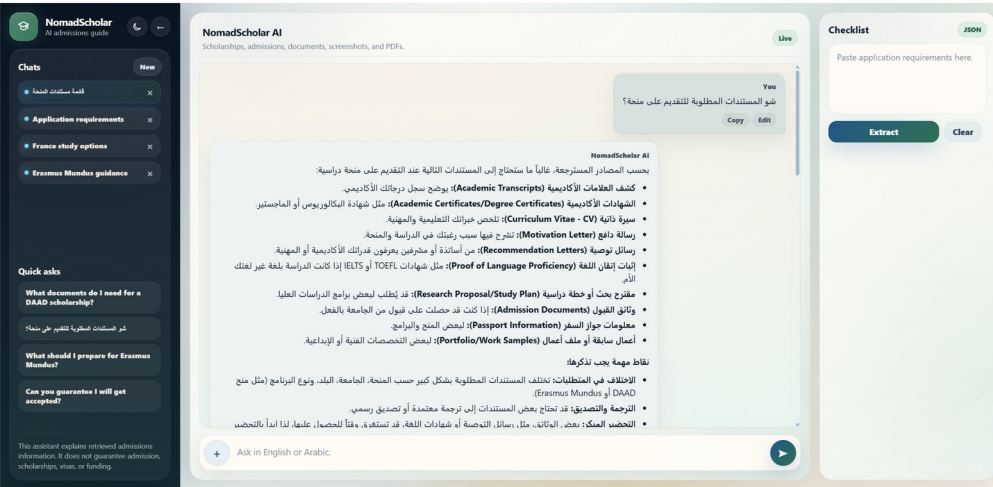


Figure 3. Arabic question and Arabic response with right-to-left formatting.

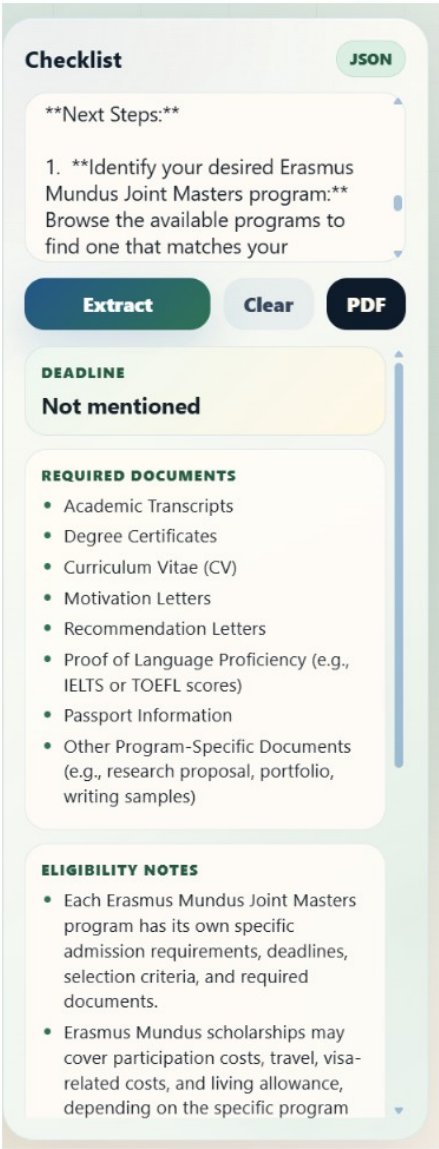


Figure 4. Cropped checklist extractor panel with structured fields and PDF export.

Bilingual support: Arabic questions receive Arabic answers with right-to-left formatting and the same safety/grounding behavior as English answers.	Function calling: the checklist extractor converts requirement text into structured fields and supports exporting the checklist as a PDF.
---	---